

Guard the Dumplings

Guard the Dumplings (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-22 · **Status:** awaiting instructor approval

Proposal (as submitted): *Guard the Dumplings* is a **tower-defense strategy game** in a European fantasy world where players defend a kingdom guarding an exclusive dumpling recipe from an enemy kingdom. Enemies invade in **waves led by elite commanders and a final boss**; players place **tower units of different classes** (Soldier, Archer, Mage) and spend an in-game currency, "**dumpling mana**" (earned from defeating enemies), to build and upgrade defenses. A **day/night cycle** structures play: **battles occur only during the day** (no building/upgrading), while **night is the strategic phase** for enhancing units and building towers. Named load-bearing C++: a game loop managing each wave — spawning many soldiers, updating towers, controlling projectiles at a stable step; a **real-time AI pathfinding system** continuously updating enemy and player soldiers' navigation toward goals; and a **health system** processing attacks/damage and updating HP + health bars in real time. Named technical features: a **real-time enemy wave-spawning system**; **tower AI & targeting** (auto-detect enemies in range, select targets by tower-specific behaviour); a **grid-based placement + path-management system** (validates placement, controls enemy movement along predefined paths); and **adaptive AI pathfinding** for enemy and player soldiers (navigate to the nearest target, adapting to the changing landscape). Tools: a **custom map editor** (which cells allow tower placement, which paths soldiers walk, and live editing of soldier/tower HP + game data); and a **dev game-speed control** (1×/2× to accelerate waves for testing wave progression, spawning and tower behaviour). Design features: the day/night cycle; tower upgrades across the three classes; progressive waves (regular troops → captain → King + elite boss stage); and the dumpling-mana economy (spend to upgrade or buy new tower types). Team: **6 CS + 1 Design + 1 Artist**.

Note (module completeness): the proposal doesn't name a **platform** — a grid-placement TD is touch-friendly, so it's planned as a mobile build (Native Android,

Pixel 9a). It also doesn't mention an **External Database** or accounts, but the module mandates Firebase — plan to persist progress / unlocked towers / level records there (a standard, small addition; see the External Database row + M1G12-style integration). *Monetization*: "dumpling mana" is an **in-game, earn-by-play** currency (clean); if real-money purchases are added, keep them cosmetic / non-pay-to-win per the module rule.

References:

- Kingdom Rush — Tower Defense on Steam (the benchmark TD with **Barracks (soldiers), Archer and Mage** tower classes, waves led by bosses, and per-tower upgrades — an almost class-for-class match to the Soldier/Archer/Mage design)
- Bloons TD 6 on Steam (place towers along predefined paths, **earn currency from defeating enemies** to buy and **upgrade** towers across branches, and survive escalating waves + boss bloons — the path-based + currency-to-upgrade + wave precedent)
- Dungeon Defenders on Steam (a TD with an explicit **build phase then combat phase** — prep and place defenses, then fight the wave — the direct precedent for the night-build / day-battle cycle)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Guard the Dumplings
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — grid placement + tapping is touch-first; a Web-on-mobile (Emscripten→WASM) build is a viable alternative. The Native PC build is the dev/tools vehicle (the map editor + the game-speed control). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for progress / unlock / level-record persistence.

Front end: A 2D top-down TD renderer — the grid map + paths, the three tower classes and their upgrades, the enemy troops/commanders/boss, projectiles, real-time health bars, and the day/night + dumpling-mana HUD.

Languages: Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; some screens (menus, upgrade shop) can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The real-time C++ engine — the wave-spawning system, tower update + AI targeting, projectile control, the adaptive A* pathfinding for enemy + player soldiers, the health/damage system, the dumpling-mana economy and the day/night phase machine — holding a stable step with many soldiers on the mobile target; plus Firebase for progress persistence. **Languages:** C++ (engine); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — the progress-persistence seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — per wave it spawns many soldiers, updates every tower's targeting + firing, advances projectiles, runs the adaptive pathfinding, and processes attacks/damage + health bars at a stable step, driving the day (battle) / night (build) phase machine.
- *Data-driven:* the tower classes + upgrade tiers, enemy troops/commanders/boss, wave compositions, dumpling-mana costs and the map layouts are all data — a new tower, enemy or wave is a data change (authored in the map/data editor).
- *2D:* the grid map + paths, towers/troops/projectiles, real-time health bars and the mana/day-night HUD (custom-rendered). *3D:* not needed.
- *Custom tools:* two — the **map editor** (placement cells, soldier paths, live soldier/tower HP + game-data editing) and the **dev game-speed control** (1×/2× to accelerate waves for testing).
- *Networking:* single-player; progress persistence is async client↔cloud (Application Communication ✓), so a Networking System is optional (●).
External DB: progress/unlocks/level records (Firebase — add per the module). *Auth » / Analytics »:* module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the wave loop + spawning + health/damage + the day/night phase machine.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep C++ loop managing wave spawning of many soldiers, tower updates, projectile control and the health/damage system, plus the day/night phase machine, at a stable step; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the AI/pathfinding (M1G8) and grid/paths (M1G14) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** An entity/component model with tower classes + upgrade tiers, enemy troops/commanders/boss, wave compositions, mana costs and map layouts in hot-reloadable JSON — a new tower, enemy or wave is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (map/data editor), M2G13 (upgrade/economy systems), M1G8 (AI data).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — incl. the **game-speed dev control**.

Tech & dependencies

- **Tech:** A dev game-speed control (1×/2×) to accelerate waves, ImGui overlays (tower ranges/targets, pathfinding, spawn counts, HP), Tracy profiling of a peak wave with many soldiers + projectiles, Catch2 tests

(targeting, pathfinding, damage math, data loading), on a GitHub Actions matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); fast wave/balance testing.

- **G4 2D Graphics** — the grid map, towers/troops/projectiles and real-time health bars.

Tech & dependencies

- **Tech: Web build:** A sprite/tile renderer for the grid map + paths, the three tower classes + upgrades, enemy troops/commanders/boss, projectiles, health bars, and the mana/day-night HUD, with a camera; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** Every visible frame; the map editor shares this renderer.

- **G8 Enemy AI & Pathfinding** — tower targeting + the **adaptive** soldier pathfinding.

Tech & dependencies

- **Tech:** Per-tower targeting (detect enemies in range, select by tower-specific rules: first/nearest/strongest) + A*/flow-field pathfinding for enemy and player soldiers toward the nearest target, re-pathing as the landscape (towers/soldiers) changes; custom C++, deterministic.
- **Prerequisites:** G1 (tick), G2 (unit data), G14 (grid/paths).
- **Feeds into:** M2G10 (advanced AI — commanders/boss, smarter targeting); the core combat.

- **G14 Level / Tile Map System** — the **grid placement + path-management** system.

Tech & dependencies

- **Tech:** A grid/tile map (JSON) marking placeable cells vs walkable paths, placement validation, and the predefined enemy paths + spatial indexing that the pathfinding queries; the format the map editor authors.
- **Prerequisites:** M1G2 (map data model), G4 (rendering).
- **Feeds into:** M2G2 (the map editor writes this format); the placement + pathing rules.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — hold the step under peak waves + the persistence seam.

Tech & dependencies

- **Tech: Web build:** Memory pools for soldiers/projectiles, Tracy profiling of a peak wave, ASan on MSVC, and on-device validation (Pixel 9a) holding the step with many units; the Firebase/Linux-server seam (progress persistence) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + stable peak-wave performance.

- **G2 ★ Content Editor — Core (Map & Data Editor)** — placement cells, soldier paths, live soldier/tower HP + game data.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: paint placeable cells vs paths, define spawn/goal points, and live-edit soldier/tower HP + game data, with round-trip JSON to the map format, live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's renderer (preview).
- **Prerequisites:** G2 (map/unit data), M1G14 (grid/path format).
- **Feeds into:** All level content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (targeting, pathfinding, damage math, wave/data loading — sped up via the game-speed control) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).

- **Feeds into:** Release/showcase builds of every goal.
- **G6 Content Editor — Advanced Tool Features (Live Data Editing + Speed Control)** — the map editor matured.

Tech & dependencies

- **Tech:** Dear ImGui advanced widgets: drag-drop map building, live in-play editing of soldier/tower stats + wave composition with immediate feedback, and the game-speed control surfaced as a gameplay/dev tool; deep undo/redo.
- **Prerequisites:** M2G2 (editor core).
- **Feeds into:** Fast wave/balance authoring; the content showcase.
- **G10 AI — Advanced (Commanders, Boss & Targeting)** — the elite commanders + King boss + smarter targeting.

Tech & dependencies

- **Tech:** Behaviour trees / FSMs for elite-commander and the King-boss behaviours (abilities, formations), matured tower-targeting priorities, and adaptive soldier tactics, all tuned from data.
- **Prerequisites:** M1G8 (AI & pathfinding foundation).
- **Feeds into:** The captain/boss wave stages; the difficulty curve.
- **G13 Gameplay Systems II** — tower **upgrade trees**, the **dumpling-mana economy** and the **day/night** progression.

Tech & dependencies

- **Tech:** C++ systems over the entity/data model: the per-class upgrade trees, the dumpling-mana earn/spend economy, the day(battle)/night(build) phase rules and wave progression (regular → captain → King), balancing-data-driven.
- **Prerequisites:** M1G2 (tower/enemy data), M1G1 (loop/economy hooks).
- **Feeds into:** The strategic depth at showcase; T2 content expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 1 Art (8) — **as proposed**, and well-matched: a tower-defense has a **contained but real art surface** (three tower classes + upgrade tiers, the enemy roster of troops/commanders/King, the European-fantasy map, projectiles/VFX, UI) that one artist can carry, while a single designer owns the FDD, the level/wave design, the dumpling-mana economy + upgrade balance and the day/night structure (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the level/wave/balance content outgrows one designer, consider a 2nd designer. *Scope note (advisory):* a stable many-unit loop + adaptive pathfinding + tower targeting + a grid/path system + a map editor is a solid, well-understood TD scope — one map, the three tower classes, a few wave types with one commander, and the map editor is the M1/M2 floor; the full upgrade trees, the King-boss + elite commanders, multiple maps and the polished day/night economy are the stretch. Adaptive re-pathing (soldiers navigating a changing landscape) is the trickiest CS bit — validate it early on one map.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the TD + day/night USP, the Soldier/Archer/Mage classes, the dumpling-mana economy, the wave/commander/boss progression; primary benchmark e.g. Kingdom Rush / Dungeon Defenders, plus a Singapore benchmark)
- M1D5 Level & Progression Design (the map(s), the predefined paths, the wave composition + difficulty curve and the day/night pacing — the heart of a TD)
- M1D6 Systems & Economy Design (the dumpling-mana source/sink, the per-class upgrade trees and tower/enemy balance — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (tower/upgrade/mana/wave balance via the editor's JSON, no code changes — critical for a TD)

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the placement/build UI, the mana/HP HUD, the upgrade shop and the day/night phase indicator, plus a wireframe baseline.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — the build/battle screens, upgrade shop, wave/day-night UI and results navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished place-and-defend + upgrade UX tested with players on the Pixel 9a.
- **Art M1:** 1 artist (towers + enemy roster + map + VFX + UI)
 - M1A1 ★ Art Bible / Proof of Concept (the European-fantasy visual direction — palette, tone, tower/enemy identity language)
 - M1A2 Character Concepts & Model Sheets (the three tower classes + upgrades, and the enemy troops / commanders / King)
 - M1A3 Environment, Props & VFX Concepts (the kingdom map + paths, projectiles and the day/night look)
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A2 Character Animation — Extensive (tower attacks + upgrades, troop walk/attack/death, commander/boss animations)
 - M2A5 VFX — Critical Effects & Transitions (projectiles, tower-fire, hits, the day↔night transition)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (battle BGM, tower-fire, hit, mana-earn, build/upgrade and wave-start cues), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for a busy wave.

• **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (battle BGM + tower-fire / hit / mana / upgrade / commander / boss SFX, ≥ 6 types), scaling with wave intensity + the day/night shift.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct per-tower-class + boss cues).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the map/data editor + the game-speed control), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*